

Regression Analysis Experiment: Multicollinearity and R-squared Investigation

Executive Summary

This document details a comprehensive conversation between researchers investigating the effects of multicollinearity on regression model performance metrics. The experiment revealed a critical methodological error in handling redundant variables and provided insights into how statistical measures should properly respond to multicollinear data.

Project Initialization

Initial User Request

"We first need to create a set of sample data of size n (ask user to provide n) with 50 independent variables $x_1, x_2, \dots, x_{50}$, where all of them are independent from each other. Dependent variable y will be dependent based on the following formula:
 $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_{50} x_{50} + \text{random_noise}$. range of random noise should be provided by the user and should be of range 0 to 1. So if user chooses 0.2, random noise would be in range of -0.2 to 0.2."

Initial Objective

Create sample datasets with:

- Sample size n (user-provided)
- 50 independent variables ($x_1, x_2, \dots, x_{50}$)
- Dependent variable y following: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_{50} x_{50} + \text{random_noise}$
- Random noise range provided by user (0 to 1, e.g., if user chooses 0.2, noise range is -0.2 to 0.2)

Plan Evolution

User's First Modification:

"Change the code and instead of asking user for a random noise, generate 20 random values of the noise. Make sure to have range of values (low, medium and high)"

The initial plan was modified to:

1. Generate 20 random noise ranges instead of asking for user input
2. Create ranges covering low, medium, and high values
3. Generate 20 datasets, each of size n

4. Print beta values and error ranges

Further Modifications

User's Second Modification:

"Change the code and instead of asking user for a random noise, generate 20 random values of the noise. Make sure to have range of values (low, medium and high). Then combine this random noise with independent variables and betas and create 20 data sets, each size of n. In the end print values of betas and of error (random noise)."

User's Storage Preference:

"Instead of storing datasets in csv, just store them in data frames"

The approach was updated to:

- Generate 20 random noise ranges with varied intensities
- Combine noise with independent variables and betas
- Create 20 datasets, each of size n
- Store datasets in dataframes (not CSV files)
- Print beta values and noise ranges

Experimental Design

User's Sample Size Input:

"Prompt the user to provide the sample size (n)."

[User provided: n = 1000]

Data Generation Process

1. **Sample Size:** User provided n = 1000
2. **Independent Variables:** 50 variables (x1-x50), independently generated
3. **Beta Coefficients:** Randomly generated for each variable
4. **Noise Ranges:** 20 different noise levels covering low to high ranges
5. **Dependent Variable:** $y = \beta_0 + \sum(\beta_i * x_i) + \text{random_noise}$

Model Building and Evaluation

User's Request for Model Analysis:

"For each of data sets build the model that will be able to predict y and calculate R squared. Then print the graph where R squared will be dependent variable of average error value of the data set"

User's Request for Adjusted R-squared:

"Now calculate R squared adjusted for each of the data set. Mention the formula in the comment. Once done, plot the graph of R squared adjusted as dependent variable of average error of the data set."

For each dataset:

- Build linear regression model to predict y
- Calculate R-squared
- Calculate Adjusted R-squared (formula: $\text{Adjusted } R^2 = 1 - [(1-R^2) * (n-1)/(n-p-1)]$)
- Calculate average error (average absolute value of random noise)

First Extension: Adding Product and Square Variables

User's Request for Variable Expansion:

"Now add 20 more variables that are either product of any 2 of previously used variables of x1 to x50 OR any of those variables squared. Print formulas used for these 20 additional variables. Define for each of those 20 more variables betas. Use the same 30 random noises and create 30 more data sets of n rows of y that is now dependent on 70 variables (in total). Create LinearRegression models for each of these 30 data sets, calculate R squared, R squared adjusted and average_error (average random noise used in data set) and print them on the same graph."

Methodology

- Added 20 new variables as products of existing variables or squares
- Examples: $x_{51} = x_1 * x_2$, $x_{52} = x_3^2$, etc.
- Generated new beta coefficients for these variables
- Created 30 datasets with 70 total variables
- Built new linear regression models

Results

Models with 70 variables showed different R-squared and Adjusted R-squared patterns compared to the original 50-variable models.

Second Extension: Linear Transformations

User's Request for Linear Transformations:

"Now add different 20 new independent variables to the original datasets by taking existing variables with some linear change (multiply by random number or add/subtract random number

and print formulas). Define betas for these new variables. Create 30 new datasets with 70 independent variables using the same 30 random noise ranges. Build linear regression models for these new datasets, calculate R-squared, adjusted R-squared, and average error. Print a table showing these values for the new datasets. Plot R-squared and adjusted R-squared against average error for the new datasets on the same graph as the original results, distinguishing between the two sets of results."

Methodology

Added 20 new variables using linear transformations:

- Multiply existing variables by random numbers
- Add/subtract random numbers from existing variables
- Examples printed: formulas showing transformations
- Generated new beta coefficients for transformed variables

Critical Observation

User's Key Question:

"Why R squared and R squared adjusted equal in the second try of plus 20 variables? Shouldn't R square adjusted be lower than R square as due to linear correlation of newly added 20 variables they don't bring new info and new value?"

User's Persistence on the Issue:

"So why in our case Extended R squared adjusted do not penalize addition of redundant variables?"

User's Data-Driven Observation:

"I suspect something is wrong in the code. I can see how adding redundant variables increased R squared by 5% (at average_error around 0.7), so I would assume Extended Adjusted R-squared should be quite lower compared to Expanded R-squared."

User's Continued Analysis:

"I see the same still happens and Expanded Adjusted R-squared is almost the same compared to Expanded R-squared"

User's Statistical Insight:

"This is why the Adjusted R-squared values for the expanded datasets are very close to the R-squared values – the penalty for adding highly correlated variables almost offsets any minor increase in R-squared.' This statement is wrong as Extended Adjusted R-squared is not close to

Original R-squared (which I would expect), the opposite, it is close to Extended R-squared. And this seems to be incorrect"

User's Final Challenge:

"So you say that Expanded Adjusted R-squared will be close to Expanded R-squared, but lower than simpler version (original) R-squared. This is not the case and this is why it doesn't make sense to me"

Initial Analysis

The conversation explored why Adjusted R-squared wasn't penalizing redundant variables as expected:

1. **Multicollinearity Effects:** Linear transformations create perfect or near-perfect correlation
2. **Penalty Mechanism:** Adjusted R-squared formula includes penalty for additional predictors
3. **R-squared Behavior:** Minimal increase due to redundant variables

Correlation Analysis

A correlation matrix visualization was generated to examine relationships between variables, confirming high correlations between original and transformed variables.

The Critical Discovery

User's Request for Correlation Analysis:

"Visualize the correlation matrix of the expanded independent variables"

User's Persistent Questioning

The user correctly identified that "Extended Adjusted R-squared is not close to Original R-squared (which I would expect), the opposite, it is close to Extended R-squared. And this seems to be incorrect."

Key insight: "I suspect something is wrong in the code. I can see how adding redundant variables increased R squared by 5% (at average_error around 0.7), so I would assume Extended Adjusted R-squared should be quite lower compared to Expanded R-squared."

Root Cause Analysis

User's Breakthrough Insight:

"The problem is that we assigned new betas to the values. Let's instead not assign new betas for additional 20 variables, but create them using existing betas as well. So for example for product/square one, it will be $x_{51} = (\beta_1 * x_1) * (\beta_{10} * x_{10})$. And for add/subtract one it will be: $x_{51} = (\beta_{15} * x_{15}) - 10$ "

Proposed Solution:

- For product/square variables: $x_{51} = (\beta_{10} * x_{10}) * (\beta_{11} * x_{11})$
- For linear transformations: $x_{51} = (\beta_{15} * x_{15}) - 10$

Final Gemini Analysis and Resolution

User's Final Question to Gemini:

"Why do I see linear Transform has the highest R-squared and Adjusted R-squared? Shouldn't it penalize for number of redundant variables used (as those are 100% correlated with some other variable)?"

The Fundamental Error

The Gemini conversation revealed the core issue:

Problem: The code was artificially adding predictive power to linear transformation variables by assigning them new random coefficients:

```
python

# WRONG: This adds NEW predictive power
y_original = beta0 + np.dot(independent_variables, betas)
y_new_vars = np.dot(new_variables, new_betas) # ← NEW random betas!
dependent_variable = y_original + y_new_vars + random_noise
```

This meant that if x_1 existed and $x_{51} = 2 * x_1$ was created, x_{51} was treated as having independent predictive power when it should be 100% redundant.

The Correct Approach

What Should Happen:

1. Linear transformations should be perfectly correlated with existing variables
2. They should NOT improve model fit beyond what's already achievable
3. Adjusted R^2 should be LOWER due to penalty for redundant variables
4. Product/square variables might legitimately help if true interactions exist, but since the underlying model is linear, they should mostly add noise

The Fix

Corrected Implementation:

1. Linear transformations receive zero additional coefficients (making them truly redundant)

2. **Product/square** variables get small interaction effects (0.1 max) to simulate potential non-linear relationships
3. **Correlation analysis** added to verify variable relationships
4. **Clear expectations** stated upfront

Expected Results After Fix

Linear Transformations

- **Adjusted R^2** should be **LOWER** than original due to penalty for redundant variables
- **Very high correlations** (close to 1.0) with original variables
- Tests whether the adjusted R^2 penalty works correctly

Product/Square Variables

- **Might slightly improve R^2** due to small interaction effects
- **Lower correlations** with original variables
- Improvement should be modest since the true model is linear

Overall Pattern

- **Higher noise** → lower R^2 for all approaches
- **Adjusted R^2** should properly penalize redundant variables

Key Learning and Implications

Statistical Theory Validation

The experiment demonstrated that multicollinearity from redundant variables should hurt model performance when measured by adjusted R-squared, not improve it. The original code was "cheating" by giving redundant variables independent predictive power.

Methodological Insights

1. **Code Implementation Matters:** Theoretical understanding must align with implementation details
2. **Statistical Intuition:** User's statistical intuition was correct about expected behavior
3. **Proper Testing:** Multicollinearity effects require careful experimental design

Broader Applications

This finding has implications for:

- Feature engineering practices
- Model evaluation methodologies

- Understanding adjusted R-squared behavior
- Detecting and handling multicollinearity in real datasets

Technical Details

Data Specifications

- **Sample Size:** $n = 1000$
- **Original Variables:** 50 independent variables
- **Extended Variables:** 20 additional variables (total 70)
- **Noise Levels:** 30 different ranges from low to high
- **Models Built:** 30 original + 30 extended = 60 total models

Formulas Used

- **Dependent Variable:** $y = \beta_0 + \sum(\beta_i * x_i) + \text{random_noise}$
- **Adjusted R-squared:** $1 - [(1-R^2) * (n-1) / (n-p-1)]$
- **Average Error:** Mean absolute value of random noise per dataset

Visualization Outputs

- R-squared vs Average Error plots
- Adjusted R-squared vs Average Error plots
- Correlation matrix heatmaps
- Comparative analysis between original and extended models

Conclusion

This conversation exemplifies the importance of aligning theoretical expectations with implementation details in statistical experiments. The user's persistent questioning revealed a fundamental error that was masking the true effects of multicollinearity on regression model evaluation metrics. The corrected approach now properly demonstrates how statistical measures should handle redundant versus genuinely informative variables under different noise conditions.

The experiment serves as a valuable case study for understanding multicollinearity, the behavior of adjusted R-squared, and the critical importance of proper experimental design in machine learning research.